

Training algorithm pseudocode

Good. Now we formalize the **entire training engine**.

This will be written as structured pseudocode that your AI agent can directly convert into C++ without asking conceptual questions.

No missing steps. No ambiguity.

PIECE 6 — COMPLETE TRAINING ALGORITHM PSEUDOCODE

Definitions

Given:

- TrainSet: list of samples
Each sample:
- (feature_vector[6], label_one_hot[4])
- ValidationSet
- NeuralNetwork model
- learning_rate α
- epochs E

HIGH-LEVEL TRAINING FLOW

FUNCTION TRAIN(model, TrainSet, ValidationSet, epochs, learning_rate)

FOR epoch = 1 TO epochs:

 Shuffle(TrainSet)

 total_loss = 0

 correct_predictions = 0

 FOR each sample in TrainSet:

 X = sample.feature_vector

```
Y = sample.label_one_hot
```

```
# Forward Pass
```

```
Y_hat = model.forward(X)
```

```
# Compute Loss
```

```
loss = CrossEntropy(Y, Y_hat)
```

```
total_loss += loss
```

```
# Accuracy tracking
```

```
predicted_class = argmax(Y_hat)
```

```
true_class = argmax(Y)
```

```
IF predicted_class == true_class:
```

```
    correct_predictions += 1
```

```
# Backpropagation + Update
```

```
model.backward(X, Y, learning_rate)
```

```
END FOR
```

```
train_accuracy = correct_predictions / TrainSet.size()
```

```
avg_loss = total_loss / TrainSet.size()
```

```
# Validation phase
```

```
val_accuracy = Evaluate(model, ValidationSet)
```

```
PRINT epoch, avg_loss, train_accuracy, val_accuracy
```

```
END FOR
```

```
END FUNCTION
```

DETAILED INTERNAL BACKPROP FLOW

Inside `model.backward(X, Y, learning_rate)`:

Step 1 — Forward (store intermediate values)

$$Z1 = X * W1 + b1$$

$$A1 = \text{ReLU}(Z1)$$

$$Z2 = A1 * W2 + b2$$

$$A2 = \text{ReLU}(Z2)$$

$$Z3 = A2 * W3 + b3$$

$$A3 = \text{Softmax}(Z3)$$

Store:

- $Z1, A1$
 - $Z2, A2$
 - $Z3, A3$
-

Step 2 — Output Layer Gradient

$$dZ3 = A3 - Y$$

$$dW3 = \text{transpose}(A2) * dZ3$$

$$db3 = dZ3$$

Step 3 — Hidden Layer 2

$$dA2 = dZ3 * \text{transpose}(W3)$$

$$dZ2 = dA2 \odot \text{ReLU_derivative}(Z2)$$

$$dW2 = \text{transpose}(A1) * dZ2$$

$$db2 = dZ2$$

Step 4 — Hidden Layer 1

$dA1 = dZ2 * \text{transpose}(W2)$

$dZ1 = dA1 \odot \text{ReLU_derivative}(Z1)$

$dW1 = \text{transpose}(X) * dZ1$

$db1 = dZ1$

Step 5 — Parameter Update

$W3 = W3 - \text{learning_rate} * dW3$

$b3 = b3 - \text{learning_rate} * db3$

$W2 = W2 - \text{learning_rate} * dW2$

$b2 = b2 - \text{learning_rate} * db2$

$W1 = W1 - \text{learning_rate} * dW1$

$b1 = b1 - \text{learning_rate} * db1$

Validation Evaluation Pseudocode

FUNCTION Evaluate(model, Dataset)

 correct = 0

 FOR each sample in Dataset:

$Y_hat = \text{model.forward}(\text{sample.X})$

 predicted = argmax(Y_hat)

 true = argmax(sample.Y)

IF predicted == true:

correct += 1

END FOR

RETURN correct / Dataset.size()

END FUNCTION

Early Stopping (Optional but Recommended)

Add:

IF val_accuracy does not improve for 10 epochs:

STOP training

Training Hyperparameters (Initial Defaults)

- epochs = 300
- learning_rate = 0.01
- batch_size = 1 (stochastic gradient descent)

Optional improvement:

- Mini-batch size = 8 or 16
-

Full System Flow (Training Mode)

1. Load dataset paths
 2. Extract features
 3. Normalize (fit on training only)
 4. Split into train/val/test
 5. Initialize model
 6. Train
 7. Save weights
 8. Evaluate on test set
-

Important Stability Rules

1. Clip softmax output to avoid $\log(0)$.
 2. Use double precision.
 3. Shuffle dataset every epoch.
 4. Do NOT update weights during validation.
 5. Reuse normalization parameters from training.
-

Time Complexity Per Epoch

Let:

- n = number of training samples
- p = number of parameters (~ 224)

Training cost per epoch:

[
 $O(n \cdot p)$
]

With $n \approx 140$

Very lightweight.